

STACKSPRINT | CEN3031

Repository Setup and Project Proposal Plan

CEN3031 - LeetCode Productivity App

September 15, 2026

Team: Santiago Ramirez, Matias Camaran, and Hussain.

Proposal status: Project name and Santiago's frontend role are confirmed; confirm remaining roles and details before submission.

1. Problem and proposed solution

A student practicing for technical interviews may practice sporadically, find it difficult to compare achievement to peers, and track their activity with disjointed tools. StackSprint will combine friendly competition, visible progress, and social discussion of solutions to encourage a daily coding-practice habit. While our users may maintain multiple activity sources their desired goal is consistent improvement towards passing technical interviews.

The solution extends the students' current repository: Weekly rankings show progress among friends, problems can be discussed after completion, and both practice activity and project development are visible on a dashboard. The intended users are students and other developers preparing for technical interviews.

Challenge alignment: Interpretation matches coding skills and visibility. Must confirm the instructor-supplied challenge statement matches before submitting.

2. Project vision

StackSprint brings teammates together to build consistent coding skills before technical interviews.

For students and early-career developers who need motivation and a clear view of their technical interview preparation, StackSprint is a social coding-practice companion that brings weekly competition, progress tracking, and discussions unlocked by verified problem completion. Unlike learning in isolation while checking separate activity trackers, StackSprint's weekly leaderboard and dashboard gamify repetitions into one cohesive learning journey.

Check vision against the provided template in the textbook before submission.

3. Team roles and collaboration

Member	Primary responsibility
Santiago Ramirez	Handles frontend design details, React components, accessibility, dashboard and leaderboard pages, and integration with the backend API.
Matias Camaran	Proposed Project Coordinator and Backend Lead. Track defined scope, task progress, API contract, database model, and authentication. Leads the development and release process.
Hussain	Proposed integrations and testing lead. Works on data-source feasibility and connector interfaces, activity integrations, automated testing, end-to-end integration checks, and monitoring risks.

Ideally, every teammate works on every feature but they own quality in different layers. All three members will review each other's pull requests, write code, update assigned tasks, and keep team members aware of decisions. Ownership ensures accountability without risking the project if someone is unavailable.

4. Scope and success criteria

Feature	Minimum deliverable and acceptance criteria
Profiles and groups	User can sign into an account and join a collaborative practice group. The user's authorized group is returned from the server upon visiting the dashboard/profile page.
Weekly leaderboard	Rankings of group members by verified problems solved that week. Design a consistent UTC boundary for the week. Repeated submissions for the same problem do not increase the count; identical scores can have the same rank.
Unlockable discussions	Posts can be read and written if signed in and discussion is unlocked by verified account problem completion. Members cannot read or write locked discussions until completion has been verified. Includes server-side API request checks. Failure cases are covered by automated tests.
Activity dashboard	View number of problems solved total and per-week history with timestamps from connected accounts. GitHub events and local IDE activity each have separate metrics from problem-solving scores to prevent misleading or gamed rankings. Missing, loading, empty, and failed-sync states.
External integrations	Begin with research into automated verification with LeetCode. Add a connector for GitHub activity after core UX is functional; consider WakaTime integration a bonus. Clearly label verification mocks so simulated functionality is never confused with the real product during a demo.

Both positive and negative test cases are covered by automated tests. Connection failures are handled clearly and designed for retry. A clear marker indicates which sources are demo fixtures; never tell a user their demo credentials successfully connected to real services.

Out of scope: Running untrusted code submitted by users. Presenting LeetCode problem descriptions. Building an online judge platform. Native iOS or Android app development. Competing with similar global platforms publicly. Initial release will focus on private groups of practicing students and professionals.

Definition of done: All acceptance criteria are met and confirmed by a teammate. All tests pass. Pull request is reviewed by another teammate and documentation is updated. Approved pull request is merged. Demonstration covers account sign-in, leaderboard score calculation, attempts to view locked/unlocked discussions, and error states on dashboard.

5. Proposed programming languages and technologies

Layer	Selection and rationale
Frontend	TypeScript, React, HTML/CSS, JavaScript when needed. Choose tooling that allows re-use between dashboard, rankings, and discussion boards.
Backend	Node.js with TypeScript. The same language will be used for the frontend and backend, which communicate through an API. Architect to be understandable by the entire team.
Data	PostgreSQL. Storage is required for users, groups, practice problems, verified LeetCode submissions, weekly scores, and restricted discussion posts. Constraints prevent duplicate completion records; verification checks whether a problem was actually completed.
Testing and delivery	Vitest and Playwright when project scaffold exists. Delivery will use GitHub Actions to trigger build and selected tests when code is committed. Git and GitHub for version control and pull request review. Public repository and GitHub Projects for task tracking.
Integration approach	Connectors will be built as independent abstractions over a source. Tokens should never be committed to Git. Use ignored environment files or approved secret storage, and a committed example file with placeholders.

6. Development and configuration management

Repository: <https://github.com/Santi852/Cen3031-Proj-1>

Main is used as the Git integration branch and substitutes for Master Branch from the assignment sheet. Teammates should create short-lived feature/fix/docs branches when contributing changes and open pull requests to merge into main. During merge review at least one other teammate must approve the change.

Control	Team practice
Change workflow	Create a short-lived branch, make a focused change, and open a pull request. Obtain approval from another teammate before merging into main.
Branch protection	Main is confirmed protected. The supplied screenshot shows required pull requests, one approval, and dismissal of stale approvals. The repository owner needs to verify the detailed saved settings since the contributor account does not have permission to view them.
Environment consistency	Documentation of setup commands added when project scaffold is created. Migrations and lockfiles committed to ensure consistency. Secrets remain out of Git; include a safe environment-variable example. Review dependency changes and schema migrations via pull requests.
Review and recovery	Review comments before merging. Newly appended code to an approved pull request should be reviewed again. Use revert on a bad merge when possible. Use tags to mark agreed course milestones.
Quality checks	Manual checks will be used for planning documents. Use automated checks when application code exists. Ask repository owner to enforce passing checks before merging.
Task management	Columns are Backlog > Ready > In Progress > In Review > Done on GitHub project. Ownership is defined on each task and acceptance criteria are defined on each issue. A task cannot be moved to Done unless its work is confirmed to be complete.

7. Delivery plan

Stage	Deliverable	Lead
1 - Definition	Confirm requirements, vision, high-level roles and stack selections, integration feasibility, and backlog. Project proposal.	Matias; team review
2 - Foundation	Application scaffold created, database schema designed, sign-in implemented with group model, and test suite workflow.	Santiago: frontend; Matias: backend
3 - Core workflows	Leaderboard, problem completion integration, restricted discussions, and functional dashboard.	All
4 - Integration and release	Connector interfaces, error handling, end-to-end testing, delivery demo, and project documentation.	Hussain; team review

These stages describe task ordering, not course milestones. Deadlines will be assigned using the course calendar. Teammates will review progress daily and bring blockers to the team within 24 hours. Matias is responsible for coordinating priority changes.

8. Risk management process

Identify: Review risk factors during project kickoff. Re-evaluate after any changes to scope. Risks should be recorded with an owner and an actionable warning sign.

Assess and prioritize: Estimate likelihood and impact from 1 (Low) to 3 (High). Multiply values to determine relative priority. 1-2: Low Priority. 3-4: Medium Priority. 6-9: High Priority. Scores are subjective and should only be used for ordering; they do not represent probabilities.

Respond: Focus efforts on high-priority risks. Avoid risk by not doing work that will not be done right within course limitations. Prepare for risks by defining behavior when the risk occurs before enabling the associated functionality. Common risks can be mitigated by allocating time to investigate during kickoff.

Monitor: Monitor risks during the regularly scheduled check-ins. Owners should provide current score, mitigation progress, and whether the warning sign has been triggered. If triggered, create or update an issue, complete the contingency plan, and reassess scope with the team. Risks affecting course deliverables are escalated to the instructor via Matias.

Risk register: feasibility and project delivery

R1: LeetCode data is unavailable or changing

Impact: 3 / High. **Likelihood:** 3 / High. **Priority:** 9 / High. **Owner:** Hussain. **Initial status:** Open.

Trigger: Early research cannot uncover a reliable method to confirm problem solves, source data changes, the server is down, or access is revoked.

Mitigation: Assess risks before starting features. Connectors should isolate the dependency. Normalize incoming records. Record timestamps and be aware of rate limits.

Contingency: Use demonstration fixtures that are clearly labeled; seek instructor approval if a scope change reduces features or requires an alternative verification process. Discussion posts will not be unlocked based on self-reported problem solves.

R2: Project scope will exceed available time

Impact: 3 / High. **Likelihood:** 3 / High. **Priority:** 9 / High. **Owner:** Matias. **Initial status:** Open.

Trigger: Functionality is overdue at a planned checkpoint or its relative effort exceeds the remaining capacity. Adjusted scope is approved and the feature still doesn't meet the checkpoint. Demonstration or shipping falls off the projected timeline.

Mitigation: Verify understanding of the core flows and confirm acceptance criteria. Core features have been identified. Stretch goals are outside of the minimum product.

Contingency: Strip out recognized stretch goals, simplify workflow, remove unused UI. Ship something that works. Seriously.

R3: Unavailability of a teammate or skills become blockers

Impact: 3 / High. **Likelihood:** 2 / Medium. **Priority:** 6 / High. **Owner:** Matias. **Initial status:** Open.

Trigger: A blocker has been unresolved for one day or an owner misses a project checkpoint. No response from a teammate for 24 hours. Blocked by a risk being investigated.

Mitigation: Documentation makes setup repeatable by anyone. Pair on risky features. Keep changes small enough for a teammate to review.

Contingency: Assign another owner. Pair with a teammate to resolve or reprioritize if capacity has dropped.

R4: Merge conflicts and inconsistent environments between teammates

Impact: 2 / Medium. **Likelihood:** 2 / Medium. **Priority:** 4 / Medium. **Owner:** Santiago. **Initial status:** Open.

Trigger: Branches have significantly diverged or a teammate cannot reproduce the same build/setup behavior. A communication gap is present between teammates. A merge request takes longer than expected to resolve.

Mitigation: Use short-lived branches. Require PR review for changes. Document setup process, lockfiles, and agree on runtime versions.

Contingency: Coordinate with the owner to resolve conflicts. Consider rollback if a merge broke the project build for another teammate. Fix setup instructions to prevent future regressions.

Risk register: sensitive data and presentation quality

R5: Account or authorization data is exposed

Impact: 3 / High. **Likelihood:** 2 / Medium. **Priority:** 6 / High. **Owner:** Matias. **Initial status:** Open.

Trigger: An access-control test fails or an exposed credential appears in code/logs. Real tokens are found in committed example files or code.

Mitigation: Check authorization on the server for group/discussion data. Do not expose local secrets. Audit config examples. Review application logs.

Contingency: Disable affected functionality. Revoke exposed credentials. Repair authorization rules and test before restoring affected functionality.

R6: Users are ranked incorrectly or their activity is duplicated

Impact: 2 / Medium. **Likelihood:** 2 / Medium. **Priority:** 4 / Medium. **Owner:** Hussain. **Initial status:** Open.

Trigger: Acceptance test for ties failed or verification tests don't meet the defined criteria. Scoring algorithm is inconsistent. Week-boundary tests fail. Retries are counted multiple times.

Mitigation: Clearly define start and end of a week. Use constraints to prevent duplicate scores. Document and test assumptions about ties, retries, etc.

Contingency: Temporarily hide ranks while logs are investigated. Adjust ranks based on verified completion records. Display a correction message/log entry to users. Clarify the source of risk and resolution steps.

R7: A third-party service becomes unreachable or enforces rate limits

Impact: 2 / Medium. **Likelihood:** 2 / Medium. **Priority:** 4 / Medium. **Owner:** Hussain. **Initial status:** Open.

Trigger: Connector returned HTTP failure codes or a known rate-limit message repeatedly. Downstream service impacts release functionality due to intermittent or nonexistent connectivity. Rate limits do not permit the planned refresh frequency.

Mitigation: Cache retrieved data where possible. Show last-sync time. Implement automatic retries with limits. Build workflows that don't rely on real-time success.

Contingency: Notify users the data is stale. Pause retries until the service recovers or the rate-limit window expires, then resume manually or via a scheduled sync task. Rework proposed solutions to reduce risk likelihood.

R8: Defects discovered late in the cycle impact critical flows

Impact: 2 / Medium. **Likelihood:** 2 / Medium. **Priority:** 4 / Medium. **Owner:** Santiago. **Initial status:** Open.

Trigger: A partner cannot complete the demonstration or is blocked by a UI lacking instructions or a control. Critical tests are missing from approved software. A defect is found during user-experience review or after a soft deadline. Milestones are missed consistently.

Mitigation: Review proposed wireframes as a team. Validate the entire flow including empty states, keyboard accessibility, etc.

Contingency: Fix release-blocking defects before shipping; defer minor cosmetic issues. Remove unnecessary complexity from the sprint. Simplify the UI.

9. Submission tasks

Submit this file as a PDF with screenshots of your project repository, configuration-management process documentation, branch-protection screen, and populated project board linked in your README. Confirm remaining roles and stack before submitting. Confirm the challenge and vision template from your textbook before submission. Review the class calendar and assign task estimates before final submission.